



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The **state space** is the complete collection of all possible states of the problem and the transitions between those states. In this grid game, each valid grid position can be considered a state, while movements between neighboring cells represent transitions. The **search tree**, on the other hand, is the structure generated by a search algorithm while exploring the state space from the initial state. A state can appear more than once in a search tree because different paths may reach the same state, whereas the state space represents the actual possible configurations of the problem. The `reached` set helps the search algorithm avoid repeatedly exploring states that have already been reached.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The `reached` set keeps track of states that have already been explored by the search algorithm. Its main purpose is to prevent the agent from repeatedly visiting the same grid cells and entering unnecessary cycles. This is especially important in a grid because the agent can move back and forth between neighboring cells. Without the `reached` set, a DFS agent could repeatedly follow a cycle such as moving from one cell to another and then returning to the previous cell, causing the search to waste a large amount of time or potentially continue indefinitely. Therefore, maintaining `reached` converts the search from repeatedly exploring a tree of duplicate states into a graph-search approach. The Lab explicitly requires this for BFS, DFS and UCS.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS and DFS explore the grid in different ways. BFS explores the shallowest states first, meaning it checks all positions at one distance from the starting position before moving to positions at greater distances. Since every movement in this grid has the same cost, the first time BFS reaches a food position it has found a path containing the minimum number of movements, so BFS is guaranteed to produce an optimal path. DFS behaves differently because it

explores one branch as deeply as possible before returning and exploring another branch. As a result, DFS can follow a long or winding route to the food even when a shorter route exists. Therefore, BFS is preferred when the shortest path is required in this equal-cost grid, while DFS may find a solution faster in some situations but does not guarantee that the solution is optimal.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS generally has a much larger memory requirement than DFS because BFS stores the frontier of many nodes at the same time. Its space complexity is approximately $O(b^d)$, where b is the branching factor and d is the depth of the shallowest solution. As the search depth increases, the number of nodes stored by BFS can grow exponentially, which can become a major memory bottleneck on a 1000x1000 grid. UCS can also consume a large amount of memory because it must maintain a priority queue containing frontier states ordered by their path costs. DFS requires much less memory in comparison, with a typical space complexity of $O(bm)$, where m is the maximum search depth. DFS mainly needs to maintain the current path and its frontier rather than storing an entire broad level of the search. Therefore, BFS and UCS are more likely to suffer from memory exhaustion on very large grids, while DFS has a smaller memory requirement but sacrifices the guarantee of finding the shortest path.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.

